Lecture 4 — Graph Modeling

> Regional Level · Prerequisites: Year Curriculum Weeks 9 (bitmasks), 16 (BFS/DFS), 17 (Dijkstra, 0-1 BFS) · Estimated time: 2–2.5 hours

1. Overview

At regional level, shortest-path problems rarely hand you a graph. They describe a maze with keys, a car with a fuel tank, a robot that pays for turning, or a network with one discount coupon. The algorithm is still BFS or Dijkstra; the difficulty is building the right graph. This lecture is about **modeling**:

- choosing **states** (nodes) and **transitions** (edges),
- recognising standard augmentations: layers, resources, bitmasks, time, direction,
- choosing the cheapest correct search for the edge weights,
- multi-source search, reversed edges, and virtual nodes,
- estimating the size of the state space before you write code.

2. States and transitions

Ask: *if I froze the process at one moment, what must I write down to continue it correctly?* That snapshot is the state. For a walker in a maze it is the cell. If the walker carries keys, it is (cell, set of keys). If turning costs time, it is (cell, facing direction). If a coupon may be used once, it is (vertex, coupon used?).

Each legal action from a state is an edge, with the cost the statement assigns. The answer is then a shortest-path or reachability query in this implicit graph. You usually do not build the graph explicitly; you generate neighbours on the fly inside the search loop.

The state must be **complete**: the set of allowed moves and their costs must depend only on the state. If you find yourself needing information that is not in the state ("but did I already pass the toll?"), add it. Then multiply: number of states × transitions per state gives the work. If that exceeds roughly 10^8 for BFS or 10^7 for Dijkstra, find a smaller model.

3. Choosing the search

Edge costs	Algorithm	Time
all equal	BFS	O(V + E)

Edge costs	Algorithm	Time
all equal, many sources	multi-source BFS	O(V + E)
0 or 1	0-1 BFS with a deque	O(V + E)
small non-negative integers	bucket queue (Dial) or split edges	O(maxW · V + E)
non-negative reals or large integers	Dijkstra	O(E log V)
negative	Bellman–Ford	O(VE)

0-1 BFS deserves emphasis. Keep a deque; when relaxing an edge of weight 0 push the node to the front, weight 1 to the back. The deque always holds nodes whose tentative distances are `d` or `d + 1` in sorted order, so popping from the front behaves like Dijkstra without a heap. As in Dijkstra, a node may be pushed several times; relax with a strict `<` and it is processed correctly. For grids with 10^6 cells and many edges per cell, the missing `log` factor matters.

4. Standard augmentations

4.1 Layers: one-time options

CSES 1195 Flight Discount: find the cheapest route from city 1 to city n if exactly one flight may be taken at half price (rounded down). Make two copies of the graph: layer 0 (coupon unused) and layer 1 (coupon used). Each flight `u → v` with cost `c` gives edges `(u,0) → (v,0)` cost `c`, `(u,1) → (v,1)` cost `c`, and `(u,0) → (v,1)` cost `⌊c/2⌋`. Run Dijkstra on the 2n states. The answer is `min(dist[n][0], dist[n][1])`; with positive costs layer 1 is always at least as good, but taking the minimum is the safe habit.

With `k` coupons there are `k + 1` layers. The same idea models "you may break at most one wall", "one teleport", or "at most k free edges".

4.2 Resources: fuel, money, energy

UVa 11367 Full Tank?: cities sell fuel at different prices, roads consume fuel equal to their length, the tank holds at most `c ≤ 100` units. Find the cheapest trip. State `(city, fuel)`. Transitions: buy **one** unit (`(u, f) → (u, f + 1)` with cost `price[u]`, if `f < c`) and drive (`(u, f) → (v, f − d)` with cost 0, if `f ≥ d`). Buying one unit at a time instead of "buy any amount" keeps the out-degree small; Dijkstra discovers the right amount automatically. With 1000 cities and 101 fuel levels this is about 10^5 states.

4.3 Bitmasks: keys and doors

A grid with keys `a–f` and doors `A–F`: a door can be crossed only after its key is collected. State `(row, col, mask)` with `R·C·64` states, BFS. Moving onto a key sets its bit; moving onto a door requires its bit. Index a flat `dist` array by `(r·C + c)·64 + mask`. A related pattern: "visit all k

special cells" with k ≤ 16 becomes BFS between special cells plus a TSP-style bitmask DP (Year Curriculum Week 26).

4.4 Time and phase

When the world changes periodically (traffic lights, moving obstacles, tides), add `t mod p` to the state and allow a "wait" edge from `(cell, t)` to `(cell, t + 1 mod p)`. When edges are available only at certain times but waiting is allowed and travel times are fixed, earliest-arrival Dijkstra works directly: arriving earlier never hurts, because you can always wait (the FIFO property).

4.5 Direction

"Going straight is free, each 90° turn costs 1": state `(cell, direction)`. Straight moves have weight 0, turns weight 1, so 0-1 BFS solves it. Laser-and-mirror problems are the same model.

5. Multi-source search, reversed edges, virtual nodes

5.1 Multi-source BFS

To compute, for every cell, the distance to the nearest source, push all sources with distance 0 before the BFS starts. This is equivalent to adding a virtual super-source connected to all sources by weight-0 edges, and costs one BFS instead of one per source.

UVa 11624 Fire!: Joe is in a maze where one or more fires spread to adjacent cells every minute. Joe also moves one cell per minute and escapes by stepping out of the grid. First run multi-source BFS from all fires to compute `fireTime[cell]`. Then BFS for Joe, entering a cell only if he arrives strictly before the fire (`joeTime + 1 < fireTime`). CSES 1194 *Monsters* is the same model, with the escape path printed.

5.2 Reversed edges

"For every node `i`, find the distance to the nearest node with property P" is a multi-source problem on the **reversed** graph: reverse all edges, start BFS from every node with property P. *Codeforces 1272E Nearest Opposite Parity* (from position `i` you may jump to `i − a_i` or `i + a_i`) is solved by two multi-source BFS runs on reversed edges, one from all odd values and one from all even values.

5.3 Virtual nodes

If every one of `k` portals connects to every other, the graph has O(k²) edges. Add a hub: `portal → hub` with the portal's cost and `hub → portal` with cost 0, which gives O(k) edges. Similarly, "all stations on line L are connected" becomes one node per line. In *Codeforces 1520G To Go Or Not To Go?* the hub idea simplifies further: BFS from the start and from the end, then combine the best "start → portal" and "portal → end" values.

5.4 Non-additive costs

Kattis `getshorty` *(Get Shorty)*: each corridor multiplies your size by a factor `f ∈ [0, 1]`; maximise your size at the exit. Dijkstra works with "maximise the product" because multiplying by `f ≤ 1` never increases the value, the analogue of non-negative weights. Alternatively, use weights `−log f ≥ 0`. In general Dijkstra needs costs that are monotone along paths; if they are not, it is wrong.

6. Reference implementations

6.1 CSES 1195 — Flight Discount (Dijkstra on layered states)

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<pair<int, ll>>> adj(n + 1);
    for (int i = 0; i < m; i++) {
        int a, b; ll c;
        cin >> a >> b >> c;
        adj[a].push_back({b, c});
    }
    const ll INF = LLONG_MAX / 4;
    vector<array<ll, 2>> dist(n + 1, {INF, INF});
    priority_queue<tuple<ll, int, int>, vector<tuple<ll, int, int>>, greater<>> pq;
    dist[1][0] = 0;
    pq.push({0, 1, 0});
    while (!pq.empty()) {
        auto [d, u, used] = pq.top();
        pq.pop();
        if (d != dist[u][used]) continue;              // stale entry
        for (auto [v, c] : adj[u]) {
            if (d + c < dist[v][used]) {                // full price
                dist[v][used] = d + c;
```

```
                pq.push({dist[v][used], v, used});
            }
            if (!used && d + c / 2 < dist[v][1]) {       // use the coupon
                dist[v][1] = d + c / 2;
                pq.push({dist[v][1], v, 1});
            }
        }
    }
    cout << min(dist[n][0], dist[n][1]) << '\n';
}
```

The CSES sample (3 cities, flights `1→2:3`, `2→3:1`, `1→3:7`, `2→1:5`) prints `2`.

6.2 AtCoder ABC 176 D — Wizard in Maze (0-1 BFS)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int H, W, sr, sc, tr, tc;
    cin >> H >> W >> sr >> sc >> tr >> tc;
    sr--, sc--, tr--, tc--;
    vector<string> g(H);
    for (auto& row : g) cin >> row;
    const int INF = INT_MAX;
    vector<vector<int>> dist(H, vector<int>(W, INF));
    deque<pair<int, int>> dq;
    dist[sr][sc] = 0;
    dq.push_back({sr, sc});
    const int dr[4] = {-1, 0, 1, 0}, dc[4] = {0, 1, 0, -1};
    while (!dq.empty()) {
        auto [r, c] = dq.front();
        dq.pop_front();
        int d = dist[r][c];
        for (int k = 0; k < 4; k++) {                    // walk: weight 0
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= H || nc < 0 || nc >= W || g[nr][nc] == '#') continue;
            if (d < dist[nr][nc]) { dist[nr][nc] = d; dq.push_front({nr, nc}); }
        }
        for (int i = -2; i <= 2; i++)                    // warp: weight 1
            for (int j = -2; j <= 2; j++) {
                int nr = r + i, nc = c + j;
                if (nr < 0 || nr >= H || nc < 0 || nc >= W || g[nr][nc] == '#') continue;
                if (d + 1 < dist[nr][nc]) { dist[nr][nc] = d + 1; dq.push_back({nr, nc});
}
            }
    }
    cout << (dist[tr][tc] == INF ? -1 : dist[tr][tc]) << '\n';
}
```

All three official samples (answers 1, −1, 0) pass.

6.3 UVa 11624 — Fire! (multi-source BFS)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int T;
    cin >> T;
    const int dr[4] = {-1, 0, 1, 0}, dc[4] = {0, 1, 0, -1};
    while (T--) {
        int R, C;
        cin >> R >> C;
        vector<string> g(R);
        for (auto& row : g) cin >> row;
        const int INF = INT_MAX;
        vector<vector<int>> fire(R, vector<int>(C, INF)), joe(R, vector<int>(C, INF));
        queue<pair<int, int>> q;
        for (int i = 0; i < R; i++)
            for (int j = 0; j < C; j++)
                if (g[i][j] == 'F') { fire[i][j] = 0; q.push({i, j}); }   // all sources
        while (!q.empty()) {
            auto [r, c] = q.front(); q.pop();
            for (int k = 0; k < 4; k++) {
                int nr = r + dr[k], nc = c + dc[k];
                if (nr < 0 || nr >= R || nc < 0 || nc >= C || g[nr][nc] == '#') continue;
                if (fire[nr][nc] == INF) { fire[nr][nc] = fire[r][c] + 1; q.push({nr,
nc}); }
            }
        }
        for (int i = 0; i < R; i++)
            for (int j = 0; j < C; j++)
                if (g[i][j] == 'J') { joe[i][j] = 0; q.push({i, j}); }
        int answer = -1;
        while (!q.empty() && answer == -1) {
            auto [r, c] = q.front(); q.pop();
            for (int k = 0; k < 4; k++) {
                int nr = r + dr[k], nc = c + dc[k];
                if (nr < 0 || nr >= R || nc < 0 || nc >= C) { answer = joe[r][c] + 1;
break; }
                if (g[nr][nc] == '#' || joe[nr][nc] != INF) continue;
                if (joe[r][c] + 1 >= fire[nr][nc]) continue;          // fire is there
first
                joe[nr][nc] = joe[r][c] + 1;
                q.push({nr, nc});
            }
        }
        if (answer == -1) cout << "IMPOSSIBLE\n";
        else cout << answer << '\n';
    }
}
```

6.4 Keys and doors (BFS over (cell, mask))

Grid with `@` start, `$` exit, `#` walls, keys `a`–`f`, doors `A`–`F`. Prints the minimum number of steps or −1.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int R, C;
    cin >> R >> C;
    vector<string> g(R);
    for (auto& row : g) cin >> row;
    int sr = 0, sc = 0;
    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++)
            if (g[i][j] == '@') sr = i, sc = j;
    vector<int> dist(R * C * 64, -1);
    auto id = [&](int r, int c, int m) { return (r * C + c) * 64 + m; };
    queue<tuple<int, int, int>> q;
    dist[id(sr, sc, 0)] = 0;
    q.push({sr, sc, 0});
    const int dr[4] = {-1, 0, 1, 0}, dc[4] = {0, 1, 0, -1};
    while (!q.empty()) {
        auto [r, c, m] = q.front(); q.pop();
        int d = dist[id(r, c, m)];
        if (g[r][c] == '$') { cout << d << '\n'; return 0; }
        for (int k = 0; k < 4; k++) {
            int nr = r + dr[k], nc = c + dc[k];
            if (nr < 0 || nr >= R || nc < 0 || nc >= C) continue;
            char ch = g[nr][nc];
            if (ch == '#') continue;
            if (ch >= 'A' && ch <= 'F' && !(m >> (ch - 'A') & 1)) continue;   // locked
            int nm = m;
            if (ch >= 'a' && ch <= 'f') nm |= 1 << (ch - 'a');              // pick up
key
            if (dist[id(nr, nc, nm)] == -1) {
                dist[id(nr, nc, nm)] = d + 1;
                q.push({nr, nc, nm});
            }
        }
    }
    cout << -1 << '\n';
}
```

On the grid `@..A..$` / `.#####.` / `......a` the answer is 10: walk around to the key and then up to the exit (the door is never needed).

7. Common mistakes

- **Incomplete state**: forgetting the direction, key mask, parity or coupon flag gives wrong answers that pass samples.
- **Checking only one layer** at the goal.
- **BFS visited marking at pop time**: nodes enter the queue many times; mark when pushing.
- **Hidden negative weights**: "gain energy on this edge" breaks Dijkstra.
- **Dijkstra without the stale check** (`if (d != dist[u]) continue;`): correct but can be much slower.
- **Memory blow-up**: 10^6 cells × 64 masks × 4-byte `int` is 256 MB. Use smaller types or a smaller model.
- **Strict vs. non-strict** comparisons in races (fire vs. Joe): read the statement's rule for simultaneous arrival.
- **Recomputing BFS per source** when one multi-source BFS suffices.

8. Practice problems

- CSES 1195 — Flight Discount: https://cses.fi/problemset/task/1195
- CSES 1194 — Monsters: https://cses.fi/problemset/task/1194
- CSES 1202 — Investigation: https://cses.fi/problemset/task/1202
- AtCoder ABC 176 D — Wizard in Maze: https://atcoder.jp/contests/abc176/tasks/abc176_d
- UVa 11624 — Fire! (UVa Online Judge)
- UVa 11367 — Full Tank? (UVa Online Judge; also POJ 3635)
- Codeforces 1063B — Labyrinth: https://codeforces.com/problemset/problem/1063/B
- Codeforces 1272E — Nearest Opposite Parity: https://codeforces.com/problemset/problem/1272/E
- Codeforces 1520G — To Go Or Not To Go?: https://codeforces.com/problemset/problem/1520/G
- Codeforces 590C — Three States: https://codeforces.com/problemset/problem/590/C
- Kattis `getshorty` — Get Shorty: https://open.kattis.com/problems/getshorty

9. Further reading

- Steven Halim et al., *Competitive Programming 4*, Section 4.4 and 8.2 (SSSP variants, state-space search).
- cp-algorithms: "0-1 BFS" (https://cp-algorithms.com/graph/01_bfs.html) and "Dijkstra on sparse graphs".
- Year Curriculum Week 17 (shortest paths) and Week 26 (bitmask DP) for the building blocks used here.